

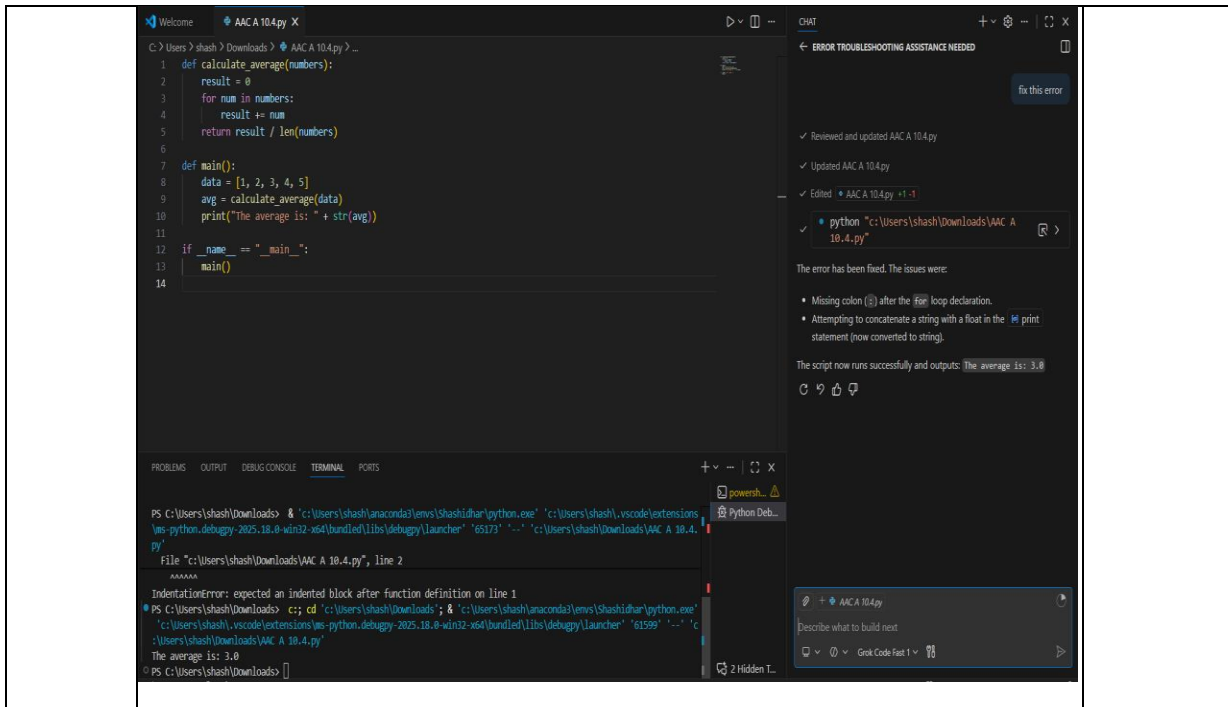
Name:G.Eshwar

H.No:2303A51808

Batch:26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																																					
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026																																				
Course Coordinator Name		Dr. Rishabh Mittal																																					
Instructor(s)Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td><td></td></tr> <tr><td>Ms. B. Swathi</td><td></td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td><td></td></tr> <tr><td>Mr. Md Sallauddin</td><td></td></tr> <tr><td>Dr. Mathivanan</td><td></td></tr> <tr><td>Mr. Y Srikanth</td><td></td></tr> <tr><td>Ms. N Shilpa</td><td></td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td><td></td></tr> <tr><td>Dr. R. Prashant Kumar</td><td></td></tr> <tr><td>Mr. Ankushavali MD</td><td></td></tr> <tr><td>Mr. B Viswanath</td><td></td></tr> <tr><td>Ms. Sujitha Reddy</td><td></td></tr> <tr><td>Ms. A. Anitha</td><td></td></tr> <tr><td>Ms. M.Madhuri</td><td></td></tr> <tr><td>Ms. Katherashala Swetha</td><td></td></tr> <tr><td>Ms. Velpula sumalatha</td><td></td></tr> <tr><td>Mr. Bingi Raju</td><td></td></tr> <tr><td>Mr. G. Kranthi</td><td></td></tr> </table>		Mr. S Naresh Kumar		Ms. B. Swathi		Dr. Sasanko Shekhar Gantayat		Mr. Md Sallauddin		Dr. Mathivanan		Mr. Y Srikanth		Ms. N Shilpa		Dr. Rishabh Mittal (Coordinator)		Dr. R. Prashant Kumar		Mr. Ankushavali MD		Mr. B Viswanath		Ms. Sujitha Reddy		Ms. A. Anitha		Ms. M.Madhuri		Ms. Katherashala Swetha		Ms. Velpula sumalatha		Mr. Bingi Raju		Mr. G. Kranthi	
Mr. S Naresh Kumar																																							
Ms. B. Swathi																																							
Dr. Sasanko Shekhar Gantayat																																							
Mr. Md Sallauddin																																							
Dr. Mathivanan																																							
Mr. Y Srikanth																																							
Ms. N Shilpa																																							
Dr. Rishabh Mittal (Coordinator)																																							
Dr. R. Prashant Kumar																																							
Mr. Ankushavali MD																																							
Mr. B Viswanath																																							
Ms. Sujitha Reddy																																							
Ms. A. Anitha																																							
Ms. M.Madhuri																																							
Ms. Katherashala Swetha																																							
Ms. Velpula sumalatha																																							
Mr. Bingi Raju																																							
Mr. G. Kranthi																																							
Course Code	23CS002PC304	Course Title	AI Assisted Coding																																				
Year/Sem	III/I	Regulation	R23																																				
Date and Day of Assignment	Week 5 - Thursday	Time(s)	23CSBTB01 To 23CSBTB52																																				
Duration	2 Hours	Applicable to Batches	All Batches																																				
AssignmentNumber: 10.4 (Present assignment number)/24(Total number of assignments)																																							
Q.No.	Question		Expected Time to complete																																				
1	Lab 9 – Code Review and Quality: Using AI to Improve Code Quality and Readability		Week 5																																				

	Lab Objectives • Use AI for automated code review and quality enhancement. • Identify and fix syntax, logical, performance, and security issues in Python code. • Improve readability and maintainability through structured refactoring and comments. • Apply prompt engineering for targeted improvements. • Evaluate AI-generated suggestions against PEP 8 standards and software engineering best practices	
	Task 1: AI-Assisted Syntax and Code Quality Review Scenario You join a development team and are asked to review a junior developer's Python script that fails to run correctly due to basic coding mistakes. Before deployment, the code must be corrected and standardized. Task Description You are given a Python script containing: • Syntax errors • Indentation issues • Incorrect variable names • Faulty function calls Use an AI tool (GitHub Copilot / Cursor AI) to: • Identify all syntactic and structural errors • Correct them systematically • Generate an explanation of each fix made Expected Outcome • Fully corrected and executable Python code • AI-generated explanation describing: ○ Syntax fixes ○ Naming corrections ○ Structural improvements • Clean, readable version of the script	



Task 2: Performance-Oriented Code Review

Scenario

A data processing function works correctly but is inefficient and slows down the system when large datasets are used.

Task Description

You are provided with a function that identifies duplicate values in a list using inefficient nested loops.

Using AI-assisted code review:

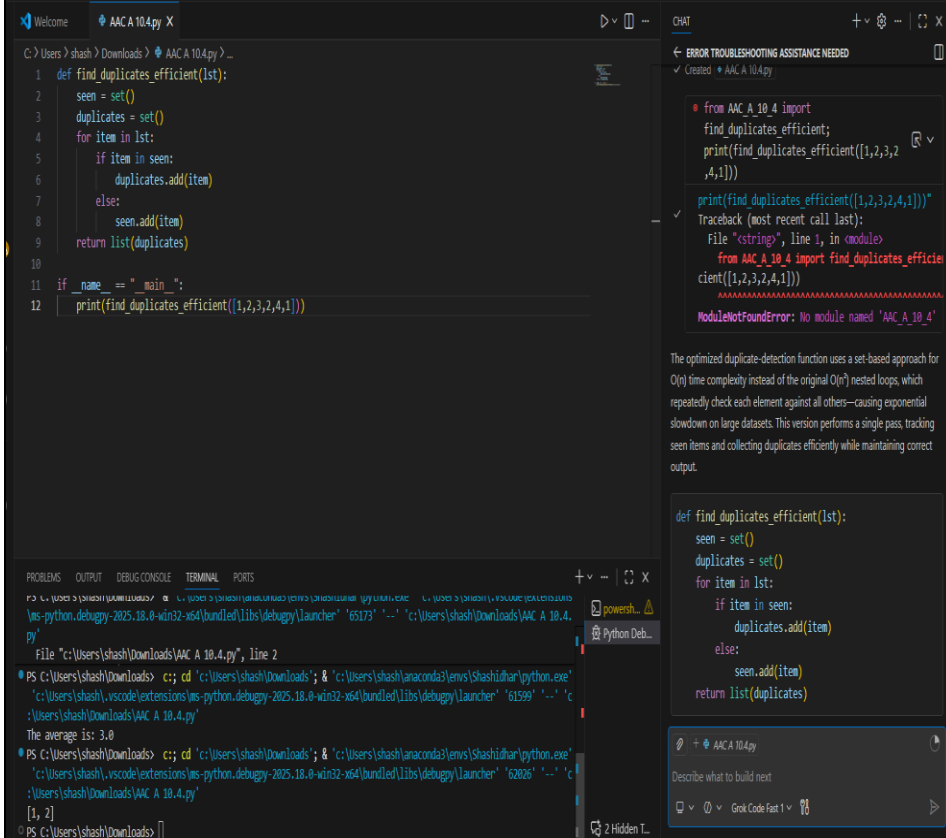
- Analyze the logic for performance bottlenecks
- Refactor the code for better time complexity
- Preserve the correctness of the output

Ask the AI to explain:

- Why the original approach was inefficient
- How the optimized version improves performance

Expected Outcome

- Optimized duplicate-detection logic (e.g., using sets or hash-based structures)
- Improved time complexity
- AI explanation of performance improvement
- Clean, readable implementation



Task 3: Readability and Maintainability Refactoring

Scenario

A working script exists in a project, but it is difficult to understand due to poor naming, formatting, and structure. The team wants it rewritten for long-term maintainability.

Task Description

You are given a poorly structured Python function with:

- Cryptic function names
- Poor indentation
- Unclear variable naming
- No documentation

Use AI-assisted review to:

- Refactor the code for clarity
- Apply PEP 8 formatting standards
- Improve naming conventions
- Add meaningful documentation

Expected Outcome

- Clean, well-structured code
- Descriptive function and variable names
- Proper indentation and formatting
- Docstrings explaining the function purpose
- AI explanation of readability improvements

The screenshot displays the VS Code interface with a Python script named `AAC A 10.4.py` and its AI-assisted refactoring process. The script defines a function `find_duplicate_values` that takes an `input_list` and returns a list of duplicates. The function uses sets to track seen values and duplicates. The main block tests the function with `[1, 2, 3, 2, 4, 1]` and prints the result.

The AI chat window on the right shows the refactoring process. It starts with a `SyntaxError: invalid syntax` and then provides a refactored version of the code. The refactored code is more descriptive, using `find_duplicate_values` instead of `find_duplicates_efficient`, and uses clearer variable names like `input_list` and `test_list`. The AI also provides a docstring for the function and explains the improvements in readability and maintainability.

```
def find_duplicate_values(input_list):
    """Find duplicate values in a list and return them as a list.

    Args:
        input_list (list): A list of integers.

    Returns:
        list: A list of duplicate values.
    """
    seen = set()
    duplicates = set()
    for item in input_list:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return list(duplicates)

if __name__ == "__main__":
    test_list = [1, 2, 3, 2, 4, 1]
    result = find_duplicate_values(test_list)
    print(result)
```

The AI chat window shows the following messages:

- Reviewed and updated AAC A 10.4.py
- Edited AAC A 10.4.py +26 -1
- python "c:\Users\shash\Downloads\AAC A 10.4.py"
- File "c:\Users\shash\Downloads\AAC A 10.4.py", line 1: `find_duplicate_values(input_list):`
- SyntaxError: invalid syntax
- Reviewed AAC A 10.4.py and updated 1 line
- Edited AAC A 10.4.py +1 -1
- python "c:\Users\shash\Downloads\AAC A 10.4.py"
- File "c:\Users\shash\Downloads\AAC A 10.4.py", line 1: `print(result)`
- SyntaxError: '(' was never closed
- Reviewed AAC A 10.4.py and modified 4 lines
- Edited AAC A 10.4.py +1 -1
- python "c:\Users\shash\Downloads\AAC A 10.4.py"

The AI chat window also shows a summary of the refactoring process:

The code has been refactored for better readability and maintainability: the function name is now descriptive (`find_duplicate_values` instead of `find_duplicates_efficient`), variable names are clearer (`input_list` vs `lst`), a docstring documents the purpose and parameters, and the main block uses meaningful variable names with proper formatting. This follows PEP 8 standards, making the code easier to understand and maintain without altering functionality.

Task 4: Secure Coding and Reliability Review

Scenario

A backend function retrieves user data from a database but has security vulnerabilities and poor error handling, making it unsafe for production deployment.

Task Description

You are given a Python script that:

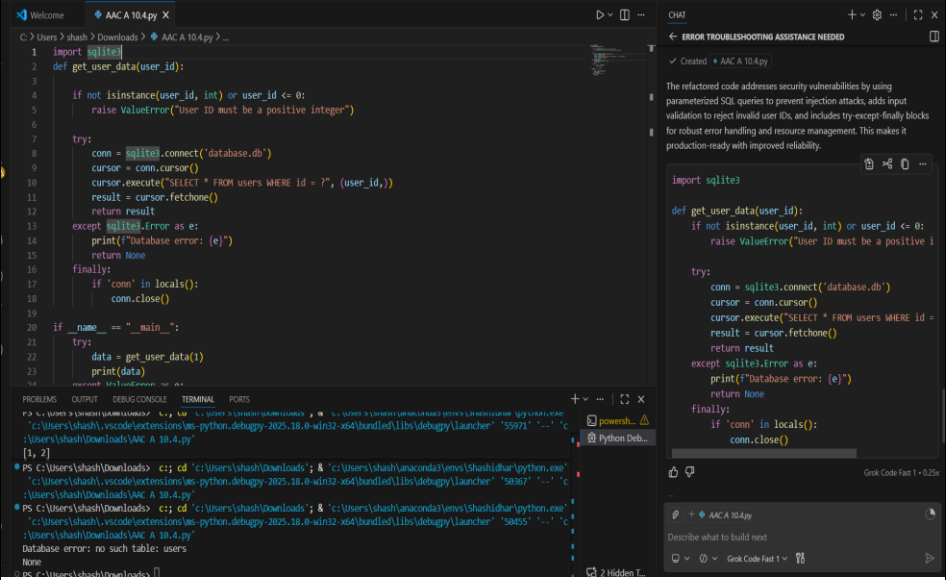
- Uses unsafe SQL query construction
- Has no input validation
- Lacks exception handling

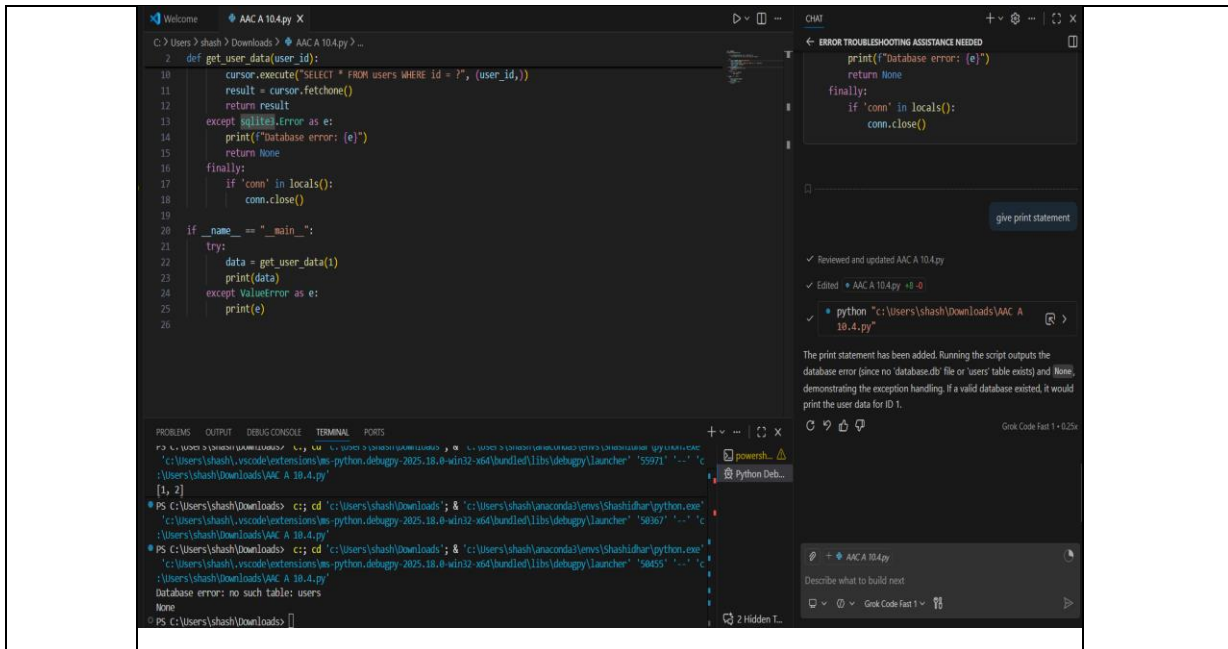
Use AI tools to:

- Identify security vulnerabilities
- Refactor the code using safe coding practices
- Add proper exception handling
- Improve robustness and reliability

Expected Outcome

- Secure SQL queries using parameterized statements
- Input validation logic
- Try-except blocks for runtime safety
- AI-generated explanation of security improvements
- Production-ready code structure





Task 5: AI-Based Automated Code Review Report

Scenario

Your team uses AI tools to perform automated preliminary code reviews before human review, to improve code quality and consistency across projects.

Task Description

You are provided with a poorly written Python script. Using AI-assisted review:

- Generate a **structured code review report** that evaluates:
 - Code readability
 - Naming conventions
 - Formatting and style consistency
 - Error handling
 - Documentation quality
 - Maintainability

The task is not just to fix the code, but to **analyze and report on quality issues**.

Expected Outcome

- AI-generated review report including:
 - Identified quality issues
 - Risk areas
 - Code smell detection
 - Improvement suggestions
- Optional improved version of the code
- Demonstration of AI as a **code reviewer**, not just a code generator

The screenshot displays a VS Code editor with a Python script named `AAC A 10.4.py`. The script attempts to connect to a SQLite database named `db` and execute a query: `SELECT * FROM users WHERE id = " + str(id)`. An error occurs: `sqlite3.OperationalError: no such table: users`. The error message is shown in the **TERMINAL** and **CHAT** panels.

The **CHAT** panel shows the error message and a suggestion to add a `try-except-finally` block for proper error handling. The **TERMINAL** panel shows the command prompt output, including the error message and the command to run the script: `python "c:\Users\shash\Downloads\AAC A 10.4.py"`.

```
1 import sqlite3
2
3 def getdata(id):
4     try:
5         conn = sqlite3.connect('db')
6         cur = conn.cursor()
7         cur.execute("SELECT * FROM users WHERE id = " + str(id))
8         res = cur.fetchone()
9         return res
10    except sqlite3.Error as e:
11        print("Database error: (e)")
12        return None
13    finally:
14        if 'conn' in locals():
15            conn.close()
16
17 if __name__ == "__main__":
18     print(getdata(1))
19
```

CHAT

ERROR TROUBLESHOOTING ASSISTANCE NEEDED

python "c:\Users\shash\Downloads\AAC A 10.4.py"

Traceback (most recent call last):

File "c:\Users\shash\Downloads\AAC A 10.4.py", line 1: print(getdata(1))

File "c:\Users\shash\Downloads\AAC A 10.4.py", line 1: cur.execute("SELECT * FROM users WHERE id = " + str(id))

sqlite3.OperationalError: no such table: users

The print statement has been added. Running the script results in a database error (no 'db' file or 'users' table exists), causing a crash as expected from the poorly written code with no error handling. The output is the traceback shown above.

fix error

Reviewed and updated AAC A 10.4.py

Edited AAC A 10.4.py +12 -6

python "c:\Users\shash\Downloads\AAC A 10.4.py"

The error has been fixed by adding try-except-finally blocks for proper error handling. The script now catches the database error, prints a

Describe what to build next.

Grak Code Fast 1

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots